

# TP0 : Analyse Exploratoire et Prétraitement des Données

**Matière :** Intelligence Artificielle

**Responsable :** Pr. S. Chabaa

**Filière :** Génie Industriel 2

## 1. Objectif du TP

Ce TP constitue une **introduction fondamentale au Machine Learning**. Avant toute application d'un algorithme d'apprentissage automatique, il est indispensable de comprendre, analyser et préparer les données.

Les objectifs principaux de ce TP sont :

- se familiariser avec le langage **Python** et les bibliothèques scientifiques usuelles,
- analyser un **signal réel mesuré** issu d'un système physique,
- détecter et corriger les **valeurs manquantes (NaN)**,
- normaliser les données afin de les rendre exploitables par les algorithmes de ML.

## 2. Chargement des bibliothèques et des données

L'objectif de cette partie est d'importer les bibliothèques nécessaires et charger le fichier de données contenant le signal mesuré. Cette étape permet d'accéder aux données sous une forme exploitable pour l'analyse et la visualisation.

### Code Python

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 data = pd.read_csv("Data.csv")
5 data_raw = data.copy()
6 t = data_raw["time"].values
7 Signal = data_raw["Signal"].values
```

### Explication ligne par ligne

- `import numpy as np` : importation de NumPy pour le calcul numérique (tableaux, opérations mathématiques).
- `import pandas as pd` : importation de Pandas pour lire le CSV et manipuler les données sous forme de DataFrame.
- `import matplotlib.pyplot as plt` : importation de Matplotlib pour tracer les courbes.
- `pd.read_csv("Data.csv")` : chargement du fichier CSV dans un DataFrame.
- `data.copy()` : création d'une copie pour conserver les données brutes intactes.
- `data_raw["time"].values` : extraction de la colonne `time` (axe temps) sous forme de tableau NumPy.
- `data_raw["Signal"].values` : extraction de la colonne `Signal` (signal mesuré) sous forme de tableau NumPy.

### 3. Analyse exploratoire (EDA) : visualisation du signal brut

Cette partie s'intéresse à observer le comportement global du signal avant tout traitement. Cette étape permet de détecter visuellement les anomalies, les ruptures ou le bruit, et de se faire une première idée de la qualité des mesures.

#### Code Python

```
1 subset = 800
2
3 t_sub = t[:subset]
4 Signal_sub = Signal[:subset]
5
6 plt.figure(figsize=(10,4))
7 plt.plot(t_sub, Signal_sub, color="blue", linewidth=1.4, label="
   Signal_brut")
8 plt.title("Signal_mesur _brut_(fenetre_partielle)")
9 plt.xlabel("Temps_(s)")
10 plt.ylabel("Amplitude")
11 plt.legend()
12 plt.grid(True)
13 plt.show()
```

### Explication ligne par ligne

- `subset = 800` : fixe le nombre d'échantillons à afficher (fenêtre partielle pour garder une figure lisible).

- `t[:subset]` : sélectionne les 800 premiers instants de temps.
- `Signal[:subset]` : sélectionne les 800 premiers échantillons du signal.
- `plt.figure(figsize=(10,4))` : crée une figure de taille 10×4 pouces.
- `plt.plot(...)` : trace le signal brut en bleu.
- `linewidth=1.4` : augmente légèrement l'épaisseur de la courbe.
- `label="Signal brut"` : associe un nom à la courbe pour la légende.
- `plt.title(...)` : ajoute un titre au graphique.
- `plt.xlabel(...)` : nomme l'axe des abscisses (temps).
- `plt.ylabel(...)` : nomme l'axe des ordonnées (amplitude).
- `plt.legend()` : affiche la légende.
- `plt.grid(True)` : active la grille pour faciliter la lecture.
- `plt.show()` : affiche la figure.

## 4. Visualisation des valeurs manquantes (NaN)

L'identification de la présence et la localisation des valeurs manquantes dans le signal seront abordés dans cette partie. En effet, les valeurs NaN peuvent fortement perturber les algorithmes de ML si elles ne sont pas traitées correctement.

### Code Python

```

1 nan_idx = np.where(np.isnan(Signal_sub))[0]
2
3 Signal_tmp = pd.Series(Signal_sub).interpolate(method="linear").
  values
4
5 plt.figure(figsize=(10,4))
6 plt.plot(t_sub, Signal_tmp, color="blue", linewidth=1.4, label="
  Signal_mesur ")
7 plt.scatter(t_sub[nan_idx], Signal_tmp[nan_idx],
8             marker="x", color="black", s=70, linewidths=2,
9             label="Valeurs_manquantes")
10
11 plt.title("Localisation_des_valeurs_manquantes")
12 plt.xlabel("Temps_(s)")
13 plt.ylabel("Amplitude")
14 plt.legend()
15 plt.grid(True)
16 plt.show()

```

### Explication ligne par ligne

- `np.isnan(Signal_sub)` : détecte les valeurs manquantes dans le signal.
- `np.where(...)[0]` : récupère les indices des valeurs manquantes.
- `pd.Series(Signal_sub)` : convertit le signal en série Pandas.
- `interpolate(method="linear")` : effectue une interpolation linéaire temporaire (uniquement pour l’affichage).
- `plt.figure(...)` : crée une figure pour la visualisation.
- `plt.plot(...)` : trace le signal interpolé de manière continue.
- `plt.scatter(...)` : marque les positions des valeurs manquantes sur la courbe.
- `marker="x"` : utilise des marqueurs en forme de croix.
- `s=70` : définit la taille des marqueurs.
- `linewidths=2` : définit l’épaisseur des marqueurs.
- `plt.title(...)` : ajoute un titre à la figure.
- `plt.legend()` : affiche la légende.
- `plt.grid(True)` : active la grille.
- `plt.show()` : affiche la figure.

**Remarque :** L’interpolation ici est uniquement pour la visualisation, les données brutes ne sont pas modifiées.

## 5. Nettoyage : correction des valeurs manquantes

Remplacer définitivement les valeurs manquantes (NaN) par des valeurs interpolées afin d’obtenir un signal continu et exploitable pour la suite du traitement.

### Code Python

```
1 data_clean = data_raw.copy()
2
3 data_clean["Signal"] = data_clean["Signal"].interpolate(method="
   linear")
4
5 Signal_clean = data_clean["Signal"].values
```

### Explication ligne par ligne

- `data_raw.copy()` : crée une copie dédiée au nettoyage.
- `data_clean["Signal"]` : sélectionne la colonne Signal à corriger.
- `interpolate(method="linear")` : remplace réellement les NaN par des valeurs interpolées.
- `data_clean["Signal"].values` : extrait le signal nettoyé sous forme de tableau NumPy.

## 6. Visualisation après interpolation

Vérifier visuellement que l'interpolation a corrigé les valeurs manquantes et que le signal traité est bien continu sans ruptures visibles.

### Code Python

```
1 plt.figure(figsize=(10,4))
2 plt.plot(t_sub, Signal_clean[:subset], color="green",
3          linewidth=1.4, label="Signal_interpol ")
4
5 plt.title("Signal après interpolation")
6 plt.xlabel("Temps(s)")
7 plt.ylabel("Amplitude")
8 plt.legend()
9 plt.grid(True)
10 plt.show()
```

### Explication ligne par ligne

- `plt.figure(figsize=(10,4))` : crée une nouvelle figure.
- `Signal_clean[:subset]` : sélectionne la même fenêtre de 800 points.
- `plt.plot(...)` : trace le signal après correction des NaN.
- `color="green"` : utilise une couleur différente pour distinguer le signal traité.
- `plt.title(...)` : ajoute un titre au graphique.
- `plt.xlabel(...)` : définit l'axe du temps.
- `plt.ylabel(...)` : définit l'axe de l'amplitude.
- `plt.legend()` : affiche la légende.
- `plt.grid(True)` : active la grille.
- `plt.show()` : affiche la figure.

## 7. Normalisation Min-Max

Normaliser le signal corrigé dans l'intervalle  $[0, 1]$  afin de supprimer l'effet d'échelle et de rendre les données compatibles avec les algorithmes de l'intelligence artificielle.

### Code Python

```
1 Signal_norm = (
2     Signal_clean - Signal_clean.min()
3 ) / (
4     Signal_clean.max() - Signal_clean.min() )
```

### Explication ligne par ligne

- `Signal_clean.min()` : calcule la valeur minimale du signal.
- `Signal_clean.max()` : calcule la valeur maximale du signal.
- `Signal_clean - Signal_clean.min()` : translate le signal pour que la valeur minimale soit égale à 0.
- `Signal_clean.max() - Signal_clean.min()` : calcule l'étendue (max-min) du signal.
- `(...)/(max-min)` : met à l'échelle pour obtenir des valeurs comprises entre 0 et 1.

## 8. Visualisation du signal normalisé

Vérifier visuellement le résultat de la normalisation et observer l'effet du changement d'échelle sur l'amplitude du signal.

### Code Python

```
1 plt.figure(figsize=(10,4))
2 plt.plot(t_sub, Signal_norm[:subset], color="magenta",
3          linewidth=1.4, label="Signal_normalis ")
4
5 plt.title("Signal_normalis _ (0 1 )")
6 plt.xlabel("Temps_(s)")
7 plt.ylabel("Amplitude_normalis e")
8 plt.legend()
9 plt.grid(True)
10 plt.show()
```

### Explication ligne par ligne

- `plt.figure(figsize=(10,4))` : crée une nouvelle figure.
- `Signal_norm[:subset]` : sélectionne la fenêtre normalisée de 800 points.
- `plt.plot(...)` : trace le signal après normalisation.
- `color="magenta"` : utilise une couleur distincte pour visualiser l'échelle normalisée.
- `plt.title(...)` : ajoute un titre au graphique.
- `plt.xlabel(...)` : définit l'axe temps.
- `plt.ylabel(...)` : définit l'axe amplitude normalisée.
- `plt.legend()` : affiche la légende.
- `plt.grid(True)` : active la grille.
- `plt.show()` : affiche la figure.

## Exercice 2

### Objectif

Dans cet exercice, on s'intéresse à l'application de toutes les étapes de prétraitement vues précédemment à partir d'un **nouveau fichier de données**.

### Données fournies

— nouveau\_fichier.csv

### Travail demandé

1. Charger les données.
2. Visualiser le signal brut (fenêtre partielle).
3. Identifier les valeurs manquantes.
4. Corriger les valeurs manquantes.
5. Normaliser le signal.
6. Comparer visuellement le signal brut et le signal traité.

### Question de réflexion

*Pourquoi le prétraitement des données est-il indispensable avant l'application d'un algorithme de l'intelligence artificielle ?*